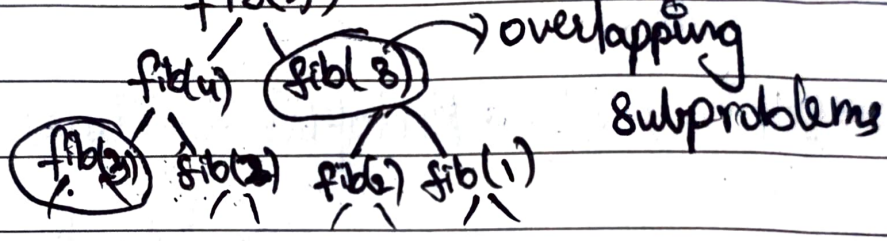
Why Dynamic Programming exists?

real Ex: Maths sheet 20 questions
question 5 → answer 42
question 12 same, 18 same → repeatedly solving same question; →

To solve this we use memoization when computes program have same problem, recursive solve smaller problems repeatedly.

Recursion trees



Memoization:

→ storing the result of function call so when same input appears again, we use stored res using Array

```
Static int fib(int n, int[] dp){
    if(n<=1){ return n; }
    if(dp[n]!=-1) return dp[n];
    dp[n]=fib(n-1,dp)+fib(n-2,dp);
    return dp[n];
}
main(){ int n=5;
    int[] dp = new int[n+1];
    Array.fill(dp,-1);
    print(fib(n,dp));
    print(Arrays.toString(dp));
}
```

reuse the existing answer

calculate the new ans

new Array to store ans

fill with(-1)

for unsolved

Date :

Four ways:

1. Tow Down DP: Memoization
   Fibonacci problem above

2. Bottom-Up DP: Tabulation
   Same problem without recursion

```
static int fib(int n){
    if(n<=1) return n;
    int[] dp = new int[n+1];
    dp[0]=0;
    dp[1]=1;          } → Base Cases
    for(int i=2; i<=n; i++){   → Build answers from small to large
        dp[i]=dp[i-1]+dp[i-2]; }
    return dp[n]; }
```

Bottom-up means we calculate smaller states first and use them to build larger states

0	Basecase	0
1	Base Case	1
2	1+0	1
3	1+1	2
4	2+1	3

Space optimization:
   We no need to store in array
In fibonacci,
   instead of array we only need prev 2 values
   current = prev1 + prev2

Date : _____

1D Problems

DP state - It is a way of describing a smaller problem whose answer we want to calculate and store.

Ex: Staircase with 5 steps, How many ways
↳ how many to reach 1?, 2?, 3? can I reach step 5?
↳ Store ans in the array called dp

what dp[i] represent: Think of i as position or index

int[] arr = {10, 20, 30, 40, 50};

↳ contains answer to the subproblem

① Ex: Sum of non-adjacent elements: arr = {2, 1, 4, 9} dp = {2, 2, 6, 11}

dp[i] = max sum we can obtain from 0 to i, without adjacent elements

dp[2] = (2, 1, 4) ⇒ 2+4=6 (1 adj element ignore)

dp[i] = max(arr[i] + dp[i-2], dp[i-1])

② MinCost climbing stairs cost = {10, 15, 20}

dp(0) = cost(0), dp(1) = cost(1) dp = {10, 15, 30}

dp[i] = cost[i] + min(dp(i-1), dp(i-2));

③ House robber

1→ current house, best from last 2 money = {2, 7, 9, 3, 1}

2→ Skip best from previous dp = {2, 7, 11, 11, 11}

Date:

dp[i] = max(current(i) + (i-2), dp(i-1));

why 2D DP?

→ when subproblem based on both row and column we use 2D-DP

int[][] dp = new int[rows][cols]

dp[i] → state indexed by single value

dp[i][j] → state indexed by two value

① Unique paths

→ robot can move either right or down

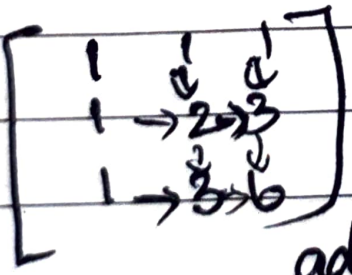


robot come from [row-1][col] → above

[row][col-1] → left

dp[r][c] = dp[r-1][c] + dp[r][c-1]

↓ no. of path to cell



0th row & 0th col & starting state 1

add values from above and left reach the bottom cell